

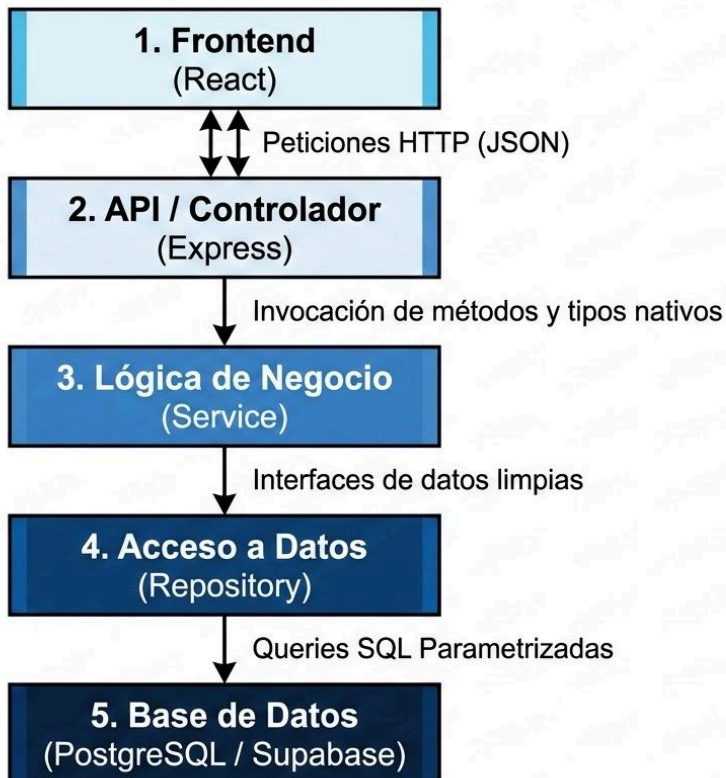
Tutorial Paso a Paso: Implementación del Mini Task Manager

Arquitectura de 5 Responsabilidades (Separation of Concerns)

Este tutorial guía la construcción integral del **Mini Task Manager** presentado en el Laboratorio 1 de Programación Web 2, materializando el principio rector: «Cada capa tiene un solo trabajo y habla exclusivamente con su vecina».

1. Mapa Arquitectónico y Estructura del Proyecto

1.1 Flujo de Información entre las 5 Capas

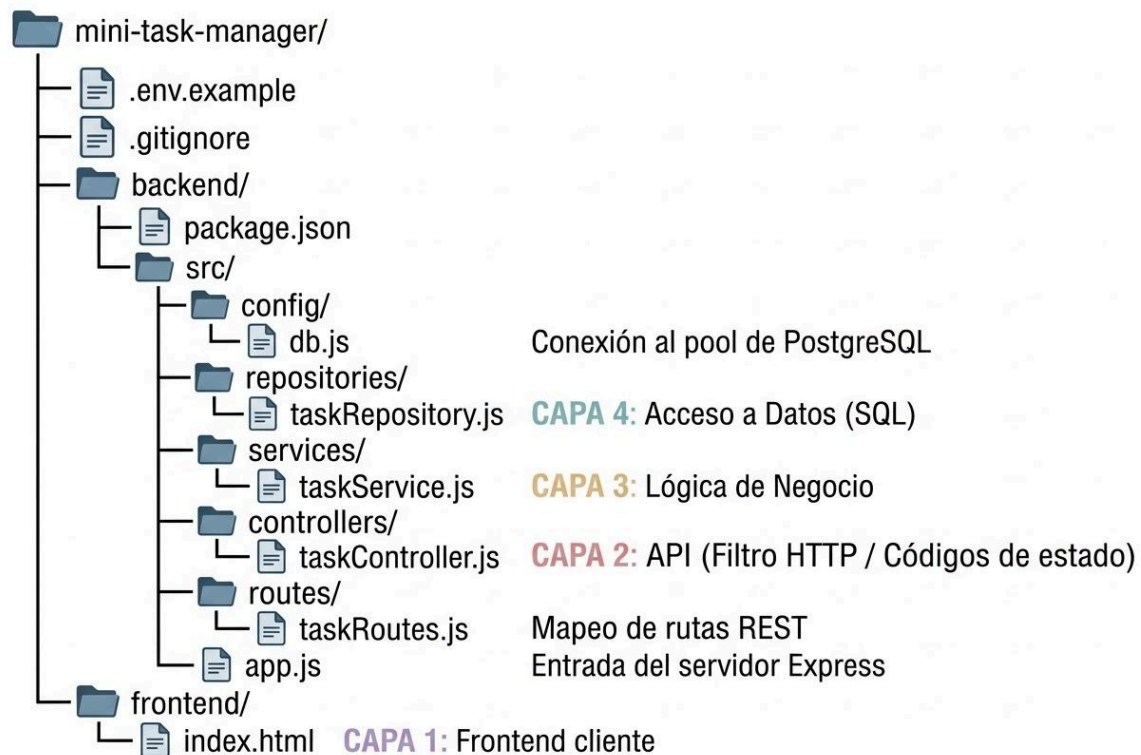


1.2 Estructura de Carpetas

Inicializa la estructura recomendada en el laboratorio ejecutando en tu terminal:

```
mkdir -p mini-task-manager/backend/src/{config,controllers,services,repositories,routes}
mkdir -p mini-task-manager/frontend/src
mkdir -p mini-task-manager/docs
cd mini-task-manager
```

El árbol resultante debe ser:



2. Capa 5: Base de Datos (PostgreSQL)

La base de datos asegura la persistencia, integridad tipada y unicidad de los registros.

Script DDL (docs/schema.sql)

Crea la tabla en tu consola de PostgreSQL o en el editor SQL de Supabase:

SQL

```
-- Habilitar extensión para UUID (opcional si usas serial, usaremos BIGSERIAL o
-- UUID)
CREATE TABLE IF NOT EXISTS tasks (
  id BIGSERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_task_status CHECK (status IN ('pending', 'done'))
);

-- Trigger para actualización automática de updated_at
CREATE OR REPLACE FUNCTION update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = CURRENT_TIMESTAMP;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trg_tasks_updated_at
BEFORE UPDATE ON tasks
FOR EACH ROW
EXECUTE FUNCTION update_timestamp();
```

3. Configuración del Backend

En la carpeta backend, inicializa Node.js e instala dependencias:

Shell

```
cd backend
npm init -y
npm install express pg cors dotenv
npm install --save-dev nodemon
```

Agrega en backend/package.json:

JSON

```
"scripts": {
  "dev": "nodemon src/app.js",
  "start": "node src/app.js"
}
```

Configura tus variables de entorno en mini-task-manager/.env:

TypeScript

```
PORT=3000
DATABASE_URL=postgresql://usuario:contraseña@localhost:5432/taskdb
```

Conexión: backend/src/config/db.js

JavaScript

```
const { Pool } = require('pg');
require('dotenv').config({ path: '../.env' });

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } :
  false
});

module.exports = {
  query: (text, params) => pool.query(text, params),
  pool
};
```

4. Capa 4: Acceso a Datos (Repository)

- **Qué hace:** Aísla el motor SQL y procesa consultas parametrizadas (\$1, \$2) contra SQL Injection (Desafío de seguridad OWASP A03).
- **Qué NO hace:** No decide reglas de negocio (ej. no evalúa si un título es grosero ni si una tarea puede reabrirse).
- **Habla con:** Lógica de negocio (arriba) y Base de datos (abajo).

backend/src/repositories/[taskRepository.js](#)

```
JavaScript
const db = require('../config/db');

class TaskRepository {
  // Obtener todas las tareas ordenadas cronológicamente
  async findAll() {
    const query = `
      SELECT id, title, status, created_at, updated_at
      FROM tasks
      ORDER BY created_at DESC;
    `;
    const { rows } = await db.query(query);
    return rows;
  }

  // Buscar tarea por ID
  async findById(id) {
    const query = `
      SELECT id, title, status, created_at, updated_at
      FROM tasks
      WHERE id = $1;
    `;
    const { rows } = await db.query(query, [id]);
    return rows[0] || null;
  }

  // Crear tarea usando marcadores parametrizados seguros ($1, $2)
  async create({ title, status }) {
    const query = `
      INSERT INTO tasks (title, status)
      VALUES ($1, $2)
      RETURNING id, title, status, created_at, updated_at;
    `;
    const { rows } = await db.query(query, [title, status]);
    return rows[0];
  }

  // Actualizar el estado de una tarea
  async updateStatus(id, newStatus) {
    const query = `
      UPDATE tasks
      SET status = $1
      WHERE id = $2
      RETURNING id, title, status, created_at, updated_at;
    `;
  }
```

```

    const { rows } = await db.query(query, [newStatus, id]);
    return rows[0] || null;
}

// Eliminar tarea físicamente
async delete(id) {
    const query = `DELETE FROM tasks WHERE id = $1 RETURNING id`;
    const { rows } = await db.query(query, [id]);
    return rows.length > 0;
}
}

module.exports = new TaskRepository();

```

5. Capa 3: Lógica de Negocio (Service)

- **Qué hace:** Aplica políticas de dominio, validaciones lógicas, normalización de datos y transiciones de estado permitidas.
- **Qué NO hace:** No sabe si fue invocada desde HTTP, gRPC o CLI; desconoce el dialecto SQL o cómo se guarda la información.
- **Habla con:** API/Controlador (arriba) y Repository (abajo).

backend/src/services/[taskService.js](#)

```

JavaScript
const taskRepository = require('../repositories/taskRepository');

class TaskService {
    async getAllTasks() {
        return await taskRepository.findAll();
    }

    async createTask(rawTitle) {
        // 1. Normalización de datos (regla de negocio)
        const sanitizedTitle = rawTitle.trim().replace(/\s+/g, ' ');

        // 2. Regla de negocio: Longitud mínima y máxima de una tarea
        if (sanitizedTitle.length < 3) {
            const error = new Error('El título de la tarea debe tener al menos 3 caracteres significativos.');
            error.statusCode = 422; // Unprocessable Entity
            error.field = 'title';
            throw error;
        }

        // 3. Regla de negocio: Moderación de contenido / política interna
        const FORBIDDEN_WORDS = ['groseria', 'spam', 'invalido'];
        const hasForbiddenWord = FORBIDDEN_WORDS.some(word =>
            sanitizedTitle.toLowerCase().includes(word)
        );
    }
}

```

```

    if (hasForbiddenWord) {
      const error = new Error('El título contiene términos no permitidos por la política de uso.');
```

política de uso.');

```

      error.statusCode = 422;
      error.field = 'title';
      throw error;
    }

    // 4. Regla: Toda nueva tarea inicia siempre como 'pending'
    const newTaskData = {
      title: sanitizedTitle,
      status: 'pending'
    };

    return await taskRepository.create(newTaskData);
  }

  async toggleTaskStatus(id) {
    const existingTask = await taskRepository.findById(id);
    if (!existingTask) {
      const error = new Error(`No se encontró la tarea con identificador ${id}.`);
      error.statusCode = 404;
      throw error;
    }

    // Regla de transición de estado: pending <=> done
    const nextStatus = existingTask.status === 'pending' ? 'done' : 'pending';
    return await taskRepository.updateStatus(id, nextStatus);
  }

  async removeTask(id) {
    const existingTask = await taskRepository.findById(id);
    if (!existingTask) {
      const error = new Error(`No se encontró la tarea con identificador ${id}.`);
      error.statusCode = 404;
      throw error;
    }

    return await taskRepository.delete(id);
  }
}

module.exports = new TaskService();

```

6. Capa 2: API (Controller y Rutas)

- **Qué hace:** Expone la interfaz REST, recibe peticiones HTTP, valida la estructura/tipado de entrada, despacha a la lógica y formula respuestas estándar con códigos de estado semánticos (200, 201, 400, 404, 500). Cumple el criterio de **accesibilidad de errores** (mensaje legible y campo asociado).
- **Qué NO hace:** No procesa SQL ni aplica reglas del negocio.
- **Habla con:** Frontend (arriba) y Lógica de Negocio (abajo).

backend/src/controllers/[taskController.js](#)

JavaScript

```
const taskService = require('../services/taskService');

class TaskController {
  async getTasks(req, res) {
    try {
      const tasks = await taskService.getAllTasks();
      return res.status(200).json({ data: tasks });
    } catch (err) {
      return res.status(500).json({
        error: 'Ocurrió un error interno al recuperar el listado de tareas.'
      });
    }
  }

  async createTask(req, res) {
    const { title } = req.body;

    // Validación de FORMATO (responsabilidad exclusiva de la Capa API)
    if (title === undefined || title === null || typeof title !== 'string') {
      return res.status(400).json({
        error: 'El campo "title" es obligatorio y debe ser una cadena de texto.',
        field: 'title'
      });
    }

    if (title.trim() === '') {
      return res.status(400).json({
        error: 'El título no puede estar vacío.',
        field: 'title'
      });
    }

    try {
      const createdTask = await taskService.createTask(title);
      return res.status(201).json({
        message: 'Tarea creada con éxito.',
        data: createdTask
      });
    } catch (err) {
      const status = err.statusCode || 500;
      return res.status(status).json({
        error: err.message,
      });
    }
  }
}
```



```

        field: err.field || null
    });
}
}

async toggleStatus(req, res) {
    const { id } = req.params;

    if (isNaN(Number(id))) {
        return res.status(400).json({
            error: 'El identificador de la tarea debe ser un valor numérico.',
            field: 'id'
        });
    }

    try {
        const updatedTask = await taskService.toggleTaskStatus(id);
        return res.status(200).json({
            message: 'Estado de tarea actualizado con éxito.',
            data: updatedTask
        });
    } catch (err) {
        return res.status(err.statusCode || 500).json({
            error: err.message
        });
    }
}

async deleteTask(req, res) {
    const { id } = req.params;

    if (isNaN(Number(id))) {
        return res.status(400).json({
            error: 'El identificador debe ser numérico.',
            field: 'id'
        });
    }

    try {
        await taskService.removeTask(id);
        return res.status(200).json({
            message: 'Tarea eliminada exitosamente.'
        });
    } catch (err) {
        return res.status(err.statusCode || 500).json({
            error: err.message
        });
    }
}
}

module.exports = new TaskController();

backend/src/routes/taskRoutes.js
const { Router } = require('express');
const taskController = require('../controllers/taskController');

```

```

const router = Router();

router.get('/tasks', (req, res) => taskController.getTasks(req, res));
router.post('/tasks', (req, res) => taskController.createTask(req, res));
router.patch('/tasks/:id/toggle', (req, res) => taskController.toggleStatus(req, res));
router.delete('/tasks/:id', (req, res) => taskController.deleteTask(req, res));

module.exports = router;

backend/src/app.js
const express = require('express');
const cors = require('cors');
const taskRoutes = require('./routes/taskRoutes');

const app = express();

// Middlewares globales
app.use(cors());
app.use(express.json());

// Montaje de rutas de la API
app.use('/api', taskRoutes);

// Ruta por defecto para recursos no encontrados
app.use((req, res) => {
  res.status(404).json({ error: 'Ruta no encontrada en la API.' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`[API Server] Servidor corriendo en http://localhost:${PORT}`);
});

```

7. Capa 1: Frontend (Cliente Web Accesible)

- **Qué hace:** Maneja la interfaz de usuario, accesibilidad (foco, etiquetas aria-, navegación por teclado), captura interacciones del usuario y consume la API vía fetch.
- **Qué NO hace:** No ejecuta consultas SQL, no aplica reglas de seguridad críticas ni almacena el estado de verdad del sistema.

Crea el archivo frontend/index.html:

```

JavaScript
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini Task Manager</title>

```

```

<style>
:root {
  --primary: #2563eb;
  --bg: #f8fafc;
  --surface: #ffffff;
  --text: #0f172a;
  --text-muted: #64748b;
  --border: #e2e8f0;
  --danger: #ef4444;
  --success: #16a34a;
}
body {
  font-family: system-ui, -apple-system, sans-serif;
  background: var(--bg);
  color: var(--text);
  display: flex;
  justify-content: center;
  padding: 2.5rem 1rem;
  margin: 0;
}
.container {
  width: 100%;
  max-width: 540px;
  background: var(--surface);
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.06);
  padding: 1.75rem;
  border: 1px solid var(--border);
}
h1 { font-size: 1.5rem; margin-top: 0; margin-bottom: 1.25rem; }
form { display: flex; gap: 0.5rem; margin-bottom: 0.5rem; }
input[type="text"] {
  flex: 1;
  padding: 0.75rem;
  border: 1px solid var(--border);
  border-radius: 6px;
  font-size: 1rem;
}
input[type="text"]:focus { outline: 2px solid var(--primary); }
button {
  padding: 0.75rem 1.25rem;
  border: none;
  border-radius: 6px;
  background: var(--primary);
  color: white;
  font-weight: 600;
  cursor: pointer;
}
.error-banner {
  color: var(--danger);
  font-size: 0.875rem;
  min-height: 1.25rem;
  margin-bottom: 1rem;
}
ul { list-style: none; padding: 0; margin: 0; }
li {
  display: flex;
  align-items: center;

```

```

        justify-content: space-between;
        padding: 0.75rem;
        border-bottom: 1px solid var(--border);
        gap: 0.75rem;
    }
    .task-title {
        flex: 1;
        cursor: pointer;
        user-select: none;
    }
    .task-title.done {
        text-decoration: line-through;
        color: var(--text-muted);
    }
    .btn-delete {
        background: transparent;
        color: var(--danger);
        padding: 0.4rem 0.6rem;
        font-size: 0.875rem;
    }
    .btn-delete:hover { background: #fee2e2; }
</style>
</head>
<body>
    <main class="container" role="main">
        <h1>Mini Task Manager</h1>

        <!-- Formulario de creación de tarea -->
        <form id="taskForm" aria-label="Crear nueva tarea">
            <input
                type="text"
                id="taskInput"
                placeholder="¿Qué tarea necesitas completar?"
                aria-label="Título de la tarea"
                aria-describedby="taskError"
                required
            />
            <button type="submit">Agregar</button>
        </form>

        <!-- Mensaje de error con soporte de accesibilidad (screen readers) -->
        <div id="taskError" class="error-banner" role="alert"
            aria-live="polite"></div>

        <!-- Lista de tareas -->
        <ul id="taskList" aria-label="Listado de tareas registradas"></ul>
    </main>

    <script>
        const API_URL = 'http://localhost:3000/api/tasks';
        const form = document.getElementById('taskForm');
        const input = document.getElementById('taskInput');
        const errorDisplay = document.getElementById('taskError');
        const taskList = document.getElementById('taskList');

        function showError(message) {
            errorDisplay.textContent = message;
        }
    </script>

```

```

function clearError() {
  errorDisplay.textContent = '';
}

async function loadTasks() {
  try {
    const res = await fetch(API_URL);
    const { data } = await res.json();
    renderTasks(data);
  } catch (err) {
    showError('No se pudo conectar con el servidor.');
```

```

  }
}

function renderTasks(tasks) {
  taskList.innerHTML = '';
  tasks.forEach(task => {
    const li = document.createElement('li');

    const span = document.createElement('span');
    span.className = `task-title ${task.status === 'done' ? 'done' : ''}`;
    span.textContent = task.title;
    span.title = 'Haz clic para alternar estado';
    span.addEventListener('click', () => toggleTask(task.id));

    const btn = document.createElement('button');
    btn.className = 'btn-delete';
    btn.textContent = 'Eliminar';
    btn.setAttribute('aria-label', `Eliminar tarea ${task.title}`);
    btn.addEventListener('click', () => deleteTask(task.id));

    li.appendChild(span);
    li.appendChild(btn);
    taskList.appendChild(li);
  });
}

```

```

form.addEventListener('submit', async (e) => {
  e.preventDefault();
  clearError();
  const title = input.value;

  try {
    const res = await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title })
    });

    const result = await res.json();

    if (!res.ok) {
      showError(result.error || 'Error al crear la tarea.');
```

```

      return;
    }
  }

  input.value = '';

```

```

        loadTasks();
    } catch (err) {
        showError('Error de red al intentar guardar la tarea.');
```

```
    }
  });

  async function toggleTask(id) {
    clearError();
    try {
      const res = await fetch(`${API_URL}/${id}/toggle`, { method: 'PATCH' });
      if (res.ok) loadTasks();
    } catch (err) {
      showError('No se pudo actualizar el estado.');
```

```
    }
  }

  async function deleteTask(id) {
    clearError();
    try {
      const res = await fetch(`${API_URL}/${id}`, { method: 'DELETE' });
      if (res.ok) loadTasks();
    } catch (err) {
      showError('No se pudo eliminar la tarea.');
```

```
    }
  }

  loadTasks();
</script>
</body>
</html>
```

8. Verificación y Pruebas del Flujo de 13 Pasos

Para verificar que cada responsabilidad responde exactamente según el diseño del laboratorio:

1. **Inicia la base de datos:** Asegúrate de que el esquema esté ejecutado en **PostgreSQL / Supabase**.
2. **Inicia el backend:**

Shell

```
cd backend
npm run dev
```

3. Prueba manual de validación de formato (API):

Envía una petición POST con cuerpo inválido:

```
Textproto
curl -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": ""}'

Respuesta esperada (400 Bad Request):
{ "error": "El título no puede estar vacío.", "field": "title" }
```

Prueba de regla de negocio (Lógica de Negocio):

Envía una petición con una palabra prohibida:

```
Textproto
curl -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "comprar spam"}'

Respuesta esperada (422 Unprocessable Entity):
{ "error": "El título contiene términos no permitidos por la política de uso.",
  "field": "title" }
```

4. Prueba de protección contra Inyección SQL (Repository):

Envía un payload malicioso:

```
Textproto
curl -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Comprar pan\'); DROP TABLE tasks;--"}'
```

Comportamiento esperado: La tarea se guarda literalmente como texto plano sin ejecutar el comando SQL destructor, gracias a los parámetros \$1 del TaskRepository.

5. **Abre el frontend:** Abre frontend/index.html en tu navegador para interactuar visualmente con el sistema completo.